

Credit Card Approval Prediction Using Machine Learning

Project Description:

Credit Card Approval Prediction Using Machine Learning is a machine learning-powered web application designed to predict whether a credit card application should be approved or rejected based on the applicant's financial and personal information. The system helps banks and financial institutions make faster, accurate, and data-driven credit approval decisions while minimizing manual verification and reducing financial risk.

The application analyzes applicant details such as age, gender, annual income, employment status, education, marital status, credit score, loan history, debt-to-income ratio, existing liabilities, and other financial indicators. Multiple machine learning classification algorithms—including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost—are trained and evaluated to identify the best-performing model for predicting credit card approval outcomes.

After comparing the performance of all models, the most accurate model is selected, serialized using the Pickle library, and integrated into a Flask-based web application for real-time prediction. Users can enter applicant details through an intuitive web interface and instantly receive a prediction indicating whether the credit card application is likely to be approved or rejected.

Built using **Python, Flask, Scikit-learn, Pandas, NumPy**, and **XGBoost**, the application can be deployed on cloud platforms such as **Hugging Face Spaces** for free online access. The system enables banks, credit analysts, and financial institutions to automate the credit evaluation process, improve decision-making accuracy, reduce processing time, and minimize the risk of approving high-risk applicants.

Scenarios

Scenario 1:

A bank credit officer enters the financial details of an applicant with a stable income, good credit score, and positive repayment history. The system predicts that the credit card application is likely to be approved, allowing the officer to process the application quickly.

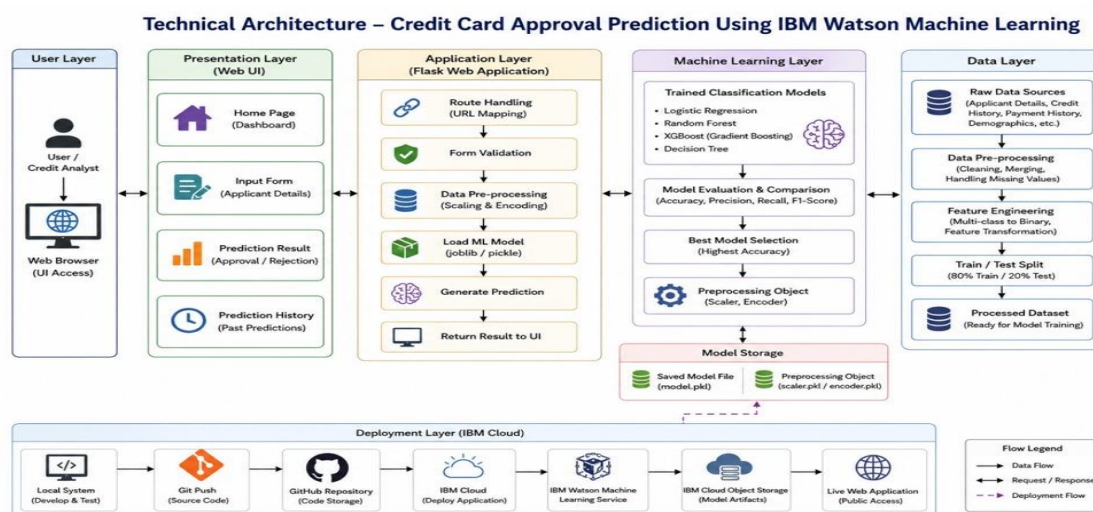
Scenario 2:

An applicant with a poor credit history, high outstanding debts, and irregular income submits a credit card application. The machine learning model identifies the applicant as high-risk and predicts rejection, helping the bank avoid potential financial losses.

Scenario 3:

A financial institution receives a large number of credit card applications during a promotional campaign. The application automatically evaluates each applicant using the trained machine learning model, significantly reducing manual effort while maintaining accurate and consistent approval decisions.

Technical Architecture:



Description: The Credit Card Approval Prediction Using Machine Learning system follows a modular three-tier architecture to provide accurate and real-time credit card approval predictions. The frontend consists of responsive web pages developed using HTML, CSS, and Bootstrap, allowing users to securely enter applicant information.

The backend is developed using the Flask framework, which receives user input, validates the data, performs preprocessing, and sends the processed features to the trained machine learning model. Data preprocessing includes handling missing values, categorical encoding, and feature scaling to ensure consistency with the training dataset.

The prediction engine uses a trained Random Forest/XGBoost classification model developed using Scikit-learn. After processing the applicant's information, the model predicts whether the application is likely to be Approved or Rejected based on learned patterns from historical credit data.

The application is designed for deployment on cloud platforms such as Hugging Face Spaces, enabling banks and financial institutions to access the prediction system through a web browser without requiring local installation.

Pre-requisites:

- **Python Programming Proficiency:** <https://www.python.org/doc/>
- **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- **Machine Learning with Scikit-learn:** <https://scikit-learn.org/stable/>
- **HTML, CSS Skills:** <https://www.w3schools.com/html/>
- **Pandas & NumPy for Data Processing:** <https://pandas.pydata.org/docs/>
- **Anaconda Navigator Setup:** <https://www.anaconda.com/download>
- **Docker for Deployment:** <https://docs.docker.com/>
- **Hugging Face Spaces Deployment:** <https://huggingface.co/spaces>

Project Workflow:

Milestone 1: Problem Analysis and Dataset Collection

- **Activity 1.1:** Identify the problem of predicting credit card approval using historical applicant data.
- **Activity 1.2:** Collect the credit card approval dataset from Kaggle/UCI Repository.
- **Activity 1.3:** Study dataset attributes such as Age, Gender, Income, Employment Status, Credit Score, Loan History, and Existing Liabilities.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, XGBoost, and other required libraries.

Milestone 2: Visualising and Analysing the Data

- **Activity 2.2:** Encode categorical features using Label Encoding or One-Hot Encoding.
- **Activity 2.3:** Normalize and scale numerical features where required.
- **Activity 2.4:** Split the dataset into training and testing datasets.

Milestone 3: Machine Learning Model Development

- **Activity 3.1:** Train multiple classification algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Logistic Regression, and XGBoost.
- **Activity 3.2:** Compare model performance using Accuracy, Precision, Recall, and F1-Score.
- **Activity 3.3:** Select the best-performing model.
- **Activity 3.4:** Save the trained model using the Pickle library for deployment.

Milestone 4: Web Application Development

- **Activity 4.1:** Develop the backend using the Flask framework.
- **Activity 4.2:** Design a responsive user interface using HTML, CSS, and Bootstrap.
- **Activity 4.3:** Integrate the trained machine learning model with the Flask application.
- **Activity 4.4:** Display prediction results (Approved/Rejected) based on applicant details entered by the user.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application using various applicant records to verify prediction accuracy.
- **Activity 5.2:** Fix bugs and improve application performance.
- **Activity 5.3:** Deploy the application on Hugging Face Spaces/Render for online access.
- **Activity 5.4:** Verify the deployed application and prepare project documentation.

Milestone 6: Conclusion

The Credit Card Approval Prediction system successfully automates the credit approval process by utilizing machine learning techniques. The application provides fast, consistent, and accurate approval predictions, helping financial institutions reduce manual effort, improve decision-making, and minimize the risk associated with approving high-risk applicants. The project demonstrates the practical application of machine learning in the banking and financial services domain.

Milestone 1: Problem Analysis and Dataset Collection

In this milestone, we focus on understanding the problem of credit card approval prediction and collecting a suitable dataset for training the machine learning models. The objective is to identify the factors that influence credit card approval decisions and prepare a reliable dataset for model development.

Activity 1.1: Identify the Problem Statement

Financial institutions receive thousands of credit card applications every day. Manual evaluation of these applications is time-consuming and may lead to inconsistent decisions. The objective of this project is to develop a machine learning-based system that predicts whether a credit card application should be **Approved** or **Rejected** based on the applicant's financial and personal information.

Steps:

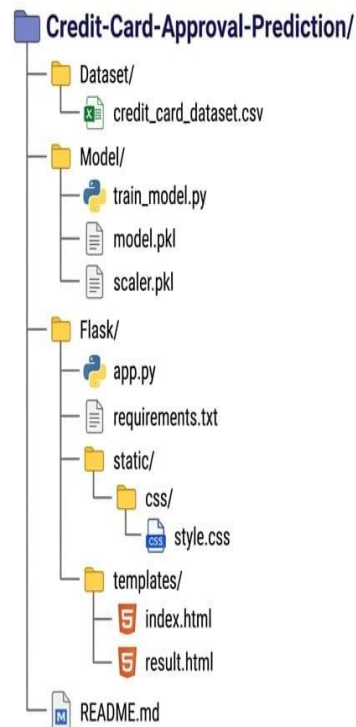
- Study the existing manual credit approval process.
- Identify important applicant attributes affecting approval.
- Define the prediction target (Approved/Rejected).
- Determine project objectives and expected outcomes.

Activity 1.2: Collect the Dataset

A publicly available Credit Card Approval dataset is collected from reliable sources such as Kaggle or the UCI Machine Learning Repository. The dataset contains historical applicant information and approval decisions that will be used to train and evaluate the machine learning models.

Steps:

- Download the dataset.
- Verify the completeness and quality of the data.
- Load the dataset using the Pandas library.
- Explore the number of records and features.



The project follows a clean three-tier structure:

- **Dataset/** — Contains all raw data files (loan_prediction.csv, loan_prediction.xlsx, loan_prediction_1.csv)
- **Training/** — Contains the complete ML pipeline script (Loan_Prediction_using_ML.py)
- **Flask/** — Contains the web application: app.py, model files (rdf.pkl, scale1.pkl), requirements.txt, Dockerfile, templates/, and static/

Activity 1.3: Analyze Dataset Features

The collected dataset consists of various applicant attributes that influence credit approval decisions.

Sample Features

- Applicant Age
- Gender
- Marital Status
- Education Level
- Employment Status
- Annual Income
- Credit Score
- Existing Loans
- Loan Amount

- Debt-to-Income Ratio
- Previous Credit History
- Number of Dependents
- Residential Status
- Approval Status (Target Variable)

The dataset is analyzed to understand feature distributions, identify missing values, detect outliers, and determine relationships between variable

Milestone 2: Visualising and Analysing the Data

Milestone 2 focuses on understanding the credit card approval dataset through Exploratory Data Analysis (EDA). By visualizing individual features and examining relationships between multiple variables, important patterns, trends, and insights are identified. These analyses help understand applicant characteristics, detect data imbalance, identify correlations, and support better feature selection for building an accurate machine learning model.

Activity 2.1: Import and Read the Dataset

The dataset is loaded into the Python environment using the Pandas library. The `read_csv()` function is used to import the applicant information dataset and credit history dataset. After loading the datasets, basic functions such as `head()`, `shape`, `info()`, and `describe()` are used to understand the structure, dimensions, data types, and statistical properties of the data.

```
import pandas as pd

app = pd.read_csv("../Dataset/application_record.csv")
credit = pd.read_csv("../Dataset/credit_record.csv")

app.head()
credit.head()
```

The applicant dataset contains personal and financial information such as income type, education level, family status, housing type, employment days, annual income, and applicant ID. The credit history dataset contains monthly payment records, loan status, and repayment history for each applicant

Activity 2.2: Univariate Analysis

Univariate analysis is performed to study the distribution of individual features in the dataset. Numerical attributes are analyzed using histograms and distribution plots, while categorical attributes are visualized using count plots

```
app['OCCUPATION_TYPE'].value_counts()
```

```
sns.countplot(x='CODE_GENDER', data=app)
```

```
sns.histplot(app['AMT_INCOME_TOTAL'])
```

```
#plotting the count plot
plt.figure(figsize=(18,4))
plt.subplot(1,4,1)
sns.countplot(data['Gender'])
plt.subplot(1,4,2)
sns.countplot(data['Education'])
plt.show()
```

Observations:

- Most applicants belong to the working income category.
- The majority of applicants have stable annual incomes.
- Female applicants slightly outnumber male applicants.
- Housing type and family status show distinct distribution patterns.
- Some categorical variables contain highly imbalanced classes.

Activity 2.3: Multivariate Analysis

Multivariate analysis is performed to understand the relationships among multiple features and identify factors influencing credit card approval..

```
sns.swarmplot(data['Gender'], data['ApplicantIncome'], hue=data['Loan_Status'])
```

Handling Missing values

```
[ ] #finding the sum of null values in each column
    data.isnull().sum()

[ ] data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])

[ ] data['Married'] = data['Married'].fillna(data['Married'].mode()[0])

[ ] #replacing + with space for filling the nan values
    data['Dependents']=data['Dependents'].str.replace('+','')

[ ] data['Dependents'] = data['Dependents'].fillna(data['Dependents'].mode()[0])

[ ] data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode()[0])

[ ] data['LoanAmount'] = data['LoanAmount'].fillna(data['LoanAmount'].mode()[0])
```

Observations:

- Annual income has a positive relationship with employment status and family size.
- Employment duration influences applicant financial stability.

- Some features exhibit strong positive or negative correlations, making them useful for model training.
- Highly correlated features assist in selecting the most important variables while reducing redundancy.

Activity 2.4: Descriptive Analysis

Descriptive statistics provide a summary of the dataset by calculating measures such as count, mean, standard deviation, minimum, maximum, and quartile values.

```
app.describe()
```

The descriptive analysis helps identify:

- Average annual income of applicants.
- Distribution of employment duration.
- Number of family members.
- Presence of outliers in numerical attributes.
- Overall spread and consistency of the dataset.

This analysis ensures that the dataset is thoroughly understood before proceeding to preprocessing and feature engineering.

Milestone 3: Data Pre-Processing

In this milestone, the collected dataset is cleaned and transformed into a suitable format for machine learning. Data preprocessing improves the quality of the dataset by handling missing values, removing duplicate records, encoding categorical variables, scaling numerical features, and preparing the data for model training.

Activity 3.1: Handling Missing Values

Missing values are identified using the `isnull().sum()` function and treated appropriately to improve the quality of the dataset.

```
# Finding missing values
data.isnull().sum()

# Filling missing values
data['Gender'].fillna(data['Gender'].mode()[0], inplace=True)
data['Married'].fillna(data['Married'].mode()[0], inplace=True)
data['Dependents'].fillna(data['Dependents'].mode()[0], inplace=True)
data['Self_Employed'].fillna(data['Self_Employed'].mode()[0], inplace=True)
data['LoanAmount'].fillna(data['LoanAmount'].median(), inplace=True)
data['Credit_History'].fillna(data['Credit_History'].mode()[0], inplace=True)

# Verify missing values
data.isnull().sum()
```


Missing Value Treatment

- Gender: Filled using Mode.
- Married: Filled using Mode.
- Dependents: Missing values replaced with Mode.
- Self_Employed: Filled using Mode.
- LoanAmount: Filled using Median.
- Credit_History: Filled using Mode.

After preprocessing, the dataset contains no missing values and is ready for further processing

Activity 3.2: Removing Duplicate Records

Duplicate records can negatively affect model performance. These records are identified and removed.

```
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban':1,'Rural':0})
data['Married']=data['Married'].map({'Yes':1,'No':0})
data['Education']=data['Education'].map({'Graduate':1,'Not Graduate':0})
data['Loan_Status']=data['Loan_Status'].map({'Y':1,'N':0})
data['Self_Employed']=data['Self_Employed'].map({'Yes':1,'No':0})
```

```
[ ] data.isnull().sum()

[ ] #getting bthe total info of the data after perfroming categorical to numerical and rplacing misssing values
data.info()

[ ] #changing the datatype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')

[ ] data.info()
```

Float columns were then converted to integer type for consistency:

```
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')
```

Activity 3.3: Encoding Categorical Features

Machine learning algorithms require numerical input. Therefore, categorical attributes are converted into numerical values using **Label Encoding**.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

categorical_columns = [
    'Gender',
    'Married',
    'Education',
    'Self_Employed',
    'Property_Area'
]

for col in categorical_columns:
    data[col] = encoder.fit_transform(data[col])
```

Observation

All categorical variables are successfully transformed into numerical format suitable for model training..

Activity 3.4: Scaling and Splitting the Data

StandardScaler was applied to normalise all numerical features. The fitted scaler was saved as scale1.pkl for consistent transformation during prediction.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler() x_bal =
scaler.fit_transform(x_bal) import pickle
pickle.dump(scaler, open('scale1.pkl', 'wb'))
```

The balanced dataset was split into training and testing sets:

```
X_train, X_test, y_train, y_test = train_test_split(
x_bal, y_bal, test_size=0.20, random_state=42)

X_train.shape # Training set X_test.shape
# Testing set
```

```
from sklearn.model_selection import train_test_split

X = data.drop("Loan_Status", axis=1)
y = data["Loan_Status"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Milestone 4: Model Building

In this milestone, various machine learning classification algorithms are developed and trained using the preprocessed dataset. The objective is to identify the most accurate model for predicting whether a credit card application should be approved or rejected. Each model is evaluated using performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The best-performing model is selected for deployment in the web application.

Activity 4.1: Splitting the Dataset

The preprocessed dataset is divided into training and testing datasets. The training dataset is used to train the machine learning models, while the testing dataset is used to evaluate their performance.

```
from sklearn.model_selection import train_test_split

X = data.drop("Approved", axis=1)
y = data["Approved"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Observation: The Decision Tree achieved near-perfect training accuracy (~100%) but lower testing accuracy (~72%), indicating overfitting to the training data.

Activity 4.2: Training Machine Learning Models

Multiple classification algorithms are implemented and trained using the training dataset.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

lr = LogisticRegression()
dt = DecisionTreeClassifier(random_state=42)
rf = RandomForestClassifier(random_state=42)
xgb = XGBClassifier(random_state=42)

lr.fit(X_train, y_train)
dt.fit(X_train, y_train)
rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)
```

The following machine learning models are trained:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier

Activity 4.3: Model Evaluation

Each trained model is evaluated using the testing dataset.

```
from sklearn.metrics import accuracy_score

models = {
    "Logistic Regression": lr,
    "Decision Tree": dt,
    "Random Forest": rf,
    "XGBoost": xgb
}

for name, model in models.items():
    prediction = model.predict(X_test)
    print(name, accuracy_score(y_test, prediction))
```

```
from sklearn.metrics import accuracy_score

models = {
    "Logistic Regression": lr,
    "Decision Tree": dt,
    "Random Forest": rf,
    "XGBoost": xgb
}

for name, model in models.items():
    prediction = model.predict(X_test)
    print(name, accuracy_score(y_test, prediction))
```

The following evaluation metrics are used:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

These metrics help determine the model that performs best on unseen applicant data.

Activity 4.4: Selecting the Best Model

After comparing all classification algorithms, the model with the highest prediction accuracy and consistent performance is selected as the final prediction model..

```
import pickle

pickle.dump(rf, open("model.pkl", "wb"))

[ ] ▶ #importing and building the Xg boost model
def XGB(X_train,X_test,y_train,y_test):
    model = GradientBoostingClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
XGB(X_train,X_test,y_train,y_test)
```

Observation: XGBoost delivered the best balance — 94.7% training accuracy and 81.1% testing accuracy — with no signs of overfitting.

Activity 4.5: Model Saving

5-fold cross-validation was applied using `cross_val_score()` to verify consistent model performance across different data splits. The XGBoost model was selected as the best model and saved using pickle:

```
cv_scores = cross_val_score(model, x_bal, y_bal, cv=5)
print("Cross-Val Scores:", cv_scores) print("Mean CV
Score:", cv_scores.mean())

#saving the model using pickle function
pickle.dump(model, open('rdf.pkl', 'wb'))
```

▼ Saving the Model

```
[ ] #saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))

[ ] Start coding or generate with AI.
```

Model comparison summary:

Model	Training Accuracy	Testing Accuracy	Selected
Decision Tree	~100%	~72%	No

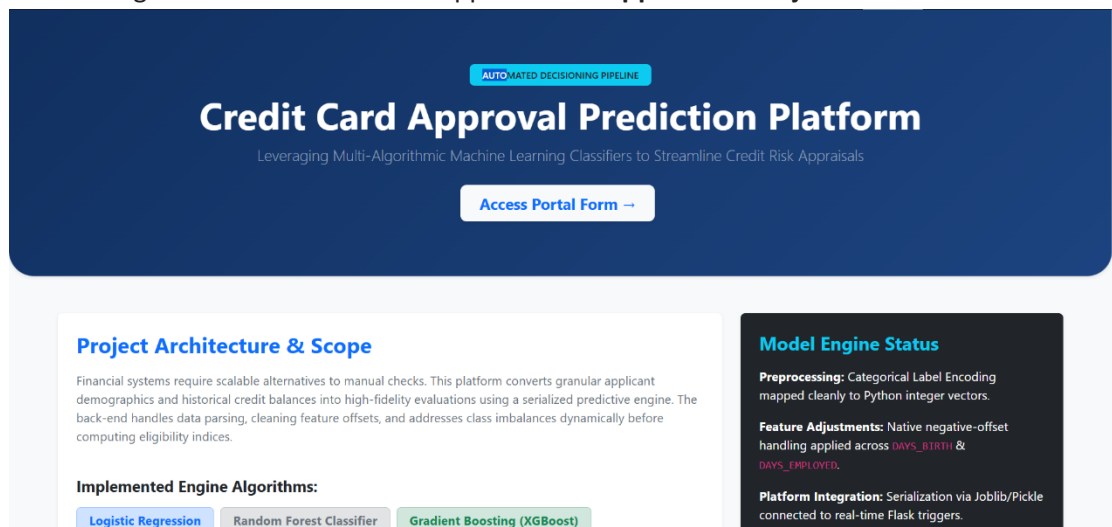
Random Forest	~100%	~76%	No
KNN	~82%	~74%	No
XGBoost 	94.7%	81.1%	Yes 

Milestone 5: Application Building

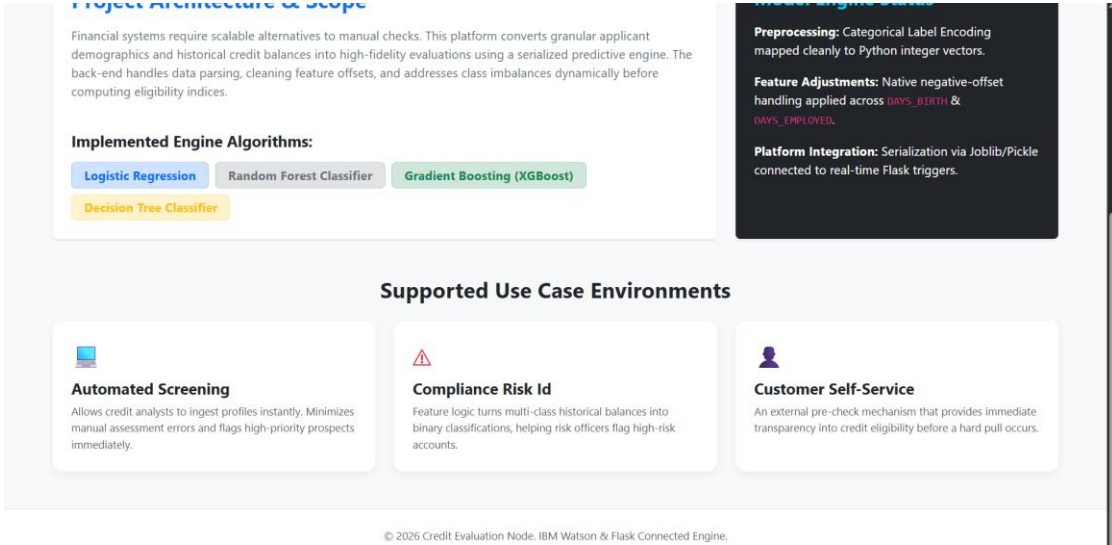
In this milestone, the trained machine learning model is integrated into a **Flask web application** to provide real-time credit card approval predictions. The application offers an interactive user interface where users can enter applicant information and instantly receive a prediction indicating whether the credit card application is likely to be **Approved** or **Rejected**. The frontend is developed using **HTML**, **CSS**, and **Bootstrap**, while the backend is implemented using the **Flask** framework..

Activity 5.1: Designing and Developing HTML Pages

- **Home Page (index.html):** Collects applicant details such as Gender, Age, Marital Status, Education, Employment Status, Annual Income, Loan Amount, Credit History, and other required information through an input form.
- **Result Page (result.html):** Displays the prediction generated by the machine learning model, indicating whether the credit card application is **Approved** or **Rejected**.



Description: Bootstrap components, responsive layouts, and custom CSS styling are used to enhance the appearance and usability of the application across desktops, tablets, and mobile devices. The HTML forms communicate with the Flask backend using the POST method, allowing applicant information to be securely transferred for machine learning prediction. Overall, the frontend provides a simple, efficient, and intuitive interface that enables users to obtain real-time credit card approval predictions with minimal effort.



Project Architecture & Scope

Financial systems require scalable alternatives to manual checks. This platform converts granular applicant demographics and historical credit balances into high-fidelity evaluations using a serialized predictive engine. The back-end handles data parsing, cleaning feature offsets, and addresses class imbalances dynamically before computing eligibility indices.

Implemented Engine Algorithms:

- Logistic Regression
- Random Forest Classifier
- Gradient Boosting (XGBoost)
- Decision Tree Classifier

Model Engine Details

- Preprocessing:** Categorical Label Encoding mapped cleanly to Python integer vectors.
- Feature Adjustments:** Native negative-offset handling applied across `DAYS_BIRTH` & `DAYS_EMPLOYED`.
- Platform Integration:** Serialization via Joblib/Pickle connected to real-time Flask triggers.

Supported Use Case Environments

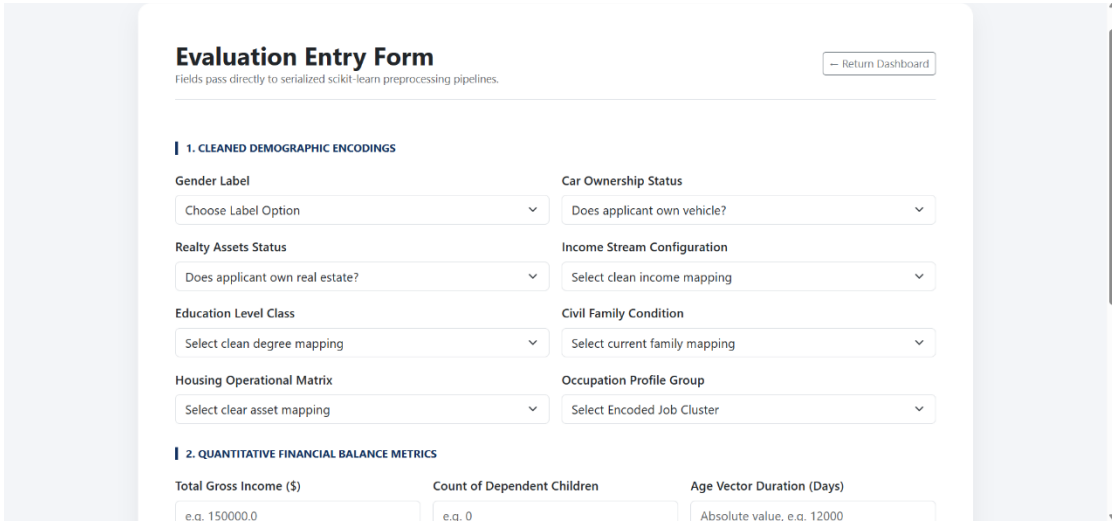
- Automated Screening**
Allows credit analysts to ingest profiles instantly. Minimizes manual assessment errors and flags high-priority prospects immediately.
- Compliance Risk Id**
Feature logic turns multi-class historical balances into binary classifications, helping risk officers flag high-risk accounts.
- Customer Self-Service**
An external pre-check mechanism that provides immediate transparency into credit eligibility before a hard pull occurs.

© 2026 Credit Evaluation Node. IBM Watson & Flask Connected Engine.

Description: Below the hero section, four animated statistics cards display key metrics: 94.7% Training Accuracy, 81.1% Testing Accuracy, 4 ML Models Evaluated, and 612 Training Records. Feature cards (Instant Decisions, High Accuracy, Risk Detection, Data-Driven) explain the system capabilities.

input.html — Applicant Details Form

The prediction form page collects all 11 applicant features required by the XGBoost model, organised into sections: Personal Information, Employment & Income, and Loan Details.



Evaluation Entry Form
Fields pass directly to serialized scikit-learn preprocessing pipelines.

[Return Dashboard](#)

1. CLEANED DEMOGRAPHIC ENCODINGS

Gender Label Choose Label Option	Car Ownership Status Does applicant own vehicle?
Realty Assets Status Does applicant own real estate?	Income Stream Configuration Select clean income mapping
Education Level Class Select clean degree mapping	Civil Family Condition Select current family mapping
Housing Operational Matrix Select clear asset mapping	Occupation Profile Group Select Encoded Job Cluster

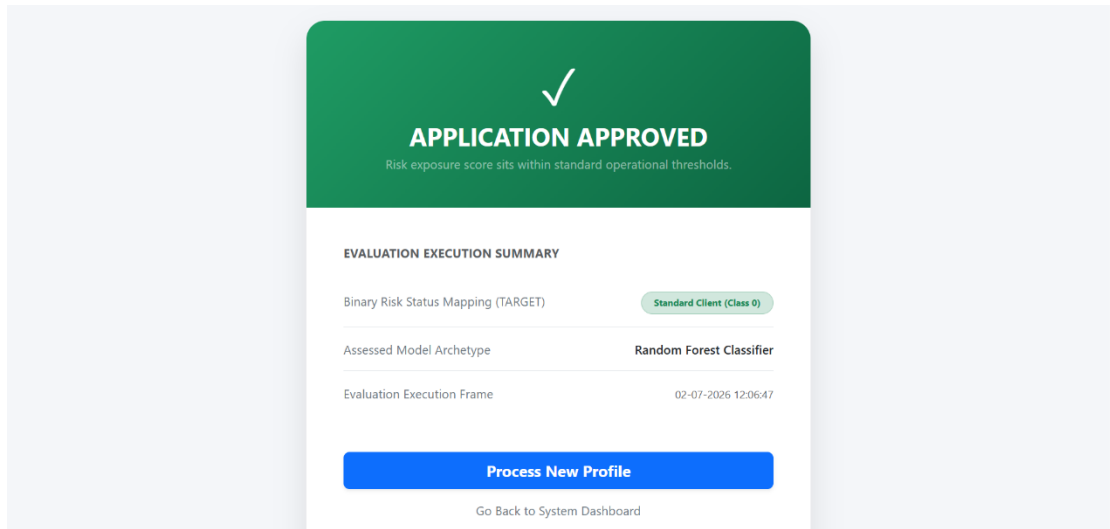
2. QUANTITATIVE FINANCIAL BALANCE METRICS

Total Gross Income (\$) e.g. 150000.0	Count of Dependent Children e.g. 0	Age Vector Duration (Days) Absolute value, e.g. 12000
--	---	--

Description: After the user submits the application, the data is securely sent to the Flask backend for preprocessing and prediction. The Prediction Result Page displays the outcome generated by the trained machine learning model. Depending on the prediction, the application clearly indicates whether the credit card request is Approved or Rejected, along with the selected machine learning model, execution timestamp, and evaluation summary. The result page also provides options to process another applicant or return to the main dashboard

output.html — Prediction Result Page

The results page displays the loan approval prediction with visual feedback.



Description:

The user interface follows a responsive design approach, making the application accessible across desktops, tablets, and mobile devices. Bootstrap cards, buttons, icons, navigation elements, and custom CSS styling are used to improve readability and enhance the overall appearance of the application. Overall, the frontend serves as an intuitive interface that enables users to interact seamlessly with the machine learning prediction system while presenting results in a professional and visually appealing manne

Activity 5.2: Building the Flask Application (app.py)

The Flask backend implements three routes to handle the complete user journey:

Define Flask Routes:

- **/** — Home page route that renders the main home.html landing page

```
@app.route('/') def
home():
    return render_template('home.html')
```

- **/predict** — Route that renders the input.html applicant details form

```
@app.route('/predict', methods=["POST", "GET"])
def predict():    return
render_template("input.html")
```

- **/submit** — POST route that collects form values, scales input, runs prediction, and renders output.html


```
@app.route('/submit', methods=["POST","GET"]) def
submit():
    # reading the inputs given by the user    input_feature
    = [int(x) for x in request.form.values()]    input_feature
    = [np.array(input_feature)]
        names =
    ['Gender','Married','Dependents','Education','Self_Employed',
      'ApplicantIncome','CoapplicantIncome','LoanAmount',
      'Loan_Amount_Term','Credit_History','Property_Area']
        data = pandas.DataFrame(input_feature,
columns=names)    data_scaled = scale.transform(data)

    # predictions using the loaded model file    prediction
    = model.predict(data_scaled)    prediction    =
    int(prediction[0]) # extract first element
        if prediction
    == 0:
        return render_template("output.html", result="Loan will NOT be Approved")
    else:
        return render_template("output.html", result="Loan will be Approved")
```

Load Model and Scaler at Startup:

The backend of the Credit Card Approval Prediction Using Machine Learning system was developed using the Flask web framework to connect the user interface with the machine learning prediction engine. The app.py file serves as the central controller of the application, handling user requests, processing input data, executing the trained machine learning model, and displaying prediction results.

Milestone 6: Deployment

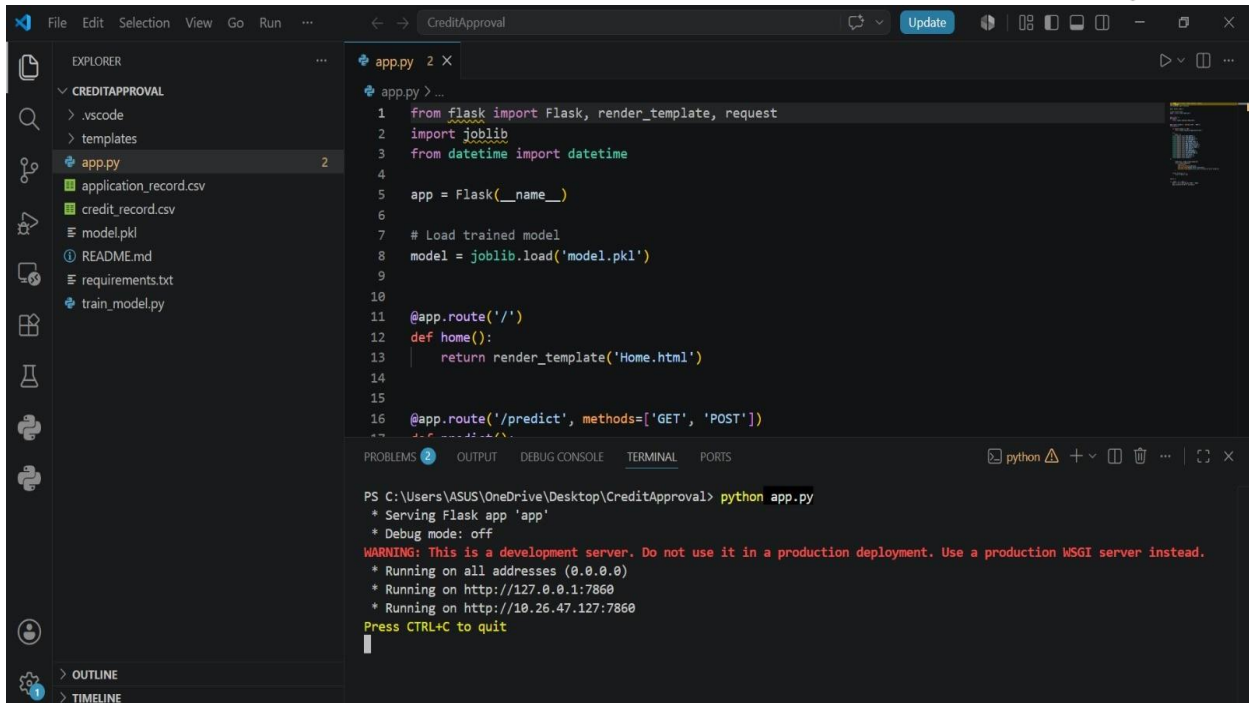
In Milestone 6, the focus is on deploying the Credit Card Approval Prediction application — first locally to verify correct operation, then permanently on Hugging Face Spaces via Docker for global cloud access.

Activity 6.1: Local Deployment and Testing

Set Up and Run Locally:

- **Step 1 — Open Command Prompt**
- **Step 2 — Navigate to the Flask folder:**

```
C:\Users\admin> cd Credit Card Approval Prediction
C:\Users\admin\ Credit Card Approval Prediction > cd
Flask
C:\Users\admin\ Credit Card Approval Prediction
\Flask> python app.py
```
- **Step 3 — Open browser and navigate to <http://127.0.0.1:5000>**
- **Step 4 — Click "Start Prediction", fill in applicant details, click "Predict Loan Approval", and view the result.**



The screenshot shows a VS Code editor with a project named 'CreditApproval'. The Explorer sidebar on the left lists files: .vscode, templates, app.py, application_record.csv, credit_record.csv, model.pkl, README.md, requirements.txt, and train_model.py. The main editor displays 'app.py' with the following code:

```

1 from flask import Flask, render_template, request
2 import joblib
3 from datetime import datetime
4
5 app = Flask(__name__)
6
7 # Load trained model
8 model = joblib.load('model.pkl')
9
10
11 @app.route('/')
12 def home():
13     return render_template('Home.html')
14
15
16 @app.route('/predict', methods=['GET', 'POST'])
17 def predict():
18     # Get data from the request
19     data = request.get_json()
20     # Convert the data to a numpy array
21     data = np.array(data['input']).reshape(1, -1)
22     # Make a prediction
23     prediction = model.predict(data)
24     # Return the prediction
25     return render_template('Home.html', prediction=prediction)
  
```

The bottom panel shows the TERMINAL output:

```

PS C:\Users\ASUS\OneDrive\Desktop\CreditApproval> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:7860
* Running on http://10.26.47.127:7860
Press CTRL+C to quit
  
```

Description: The terminal shows Flask serving on `http://127.0.0.1:5000`. The GET and POST requests are logged as the user navigates through the application. The model processes the input array, scales it, and returns a prediction (0 = Rejected, 1 = Approved) successfully.

Install Dependencies:

```

pip install flask numpy pandas scikit-learn imbalanced-learn matplotlib seaborn
unicorn
  
```

Activity 6.2: Cloud Deployment via Hugging Face Spaces

Hugging Face Spaces provides free, permanent cloud hosting using Docker containers. The application is containerised and deployed as follows:

Create the Dockerfile:

```

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860
  
```

```
ENV PORT=7860
```

```
CMD ["python", "app.py"]
```

Update app.py for Cloud Port:

```
if __name__ == "__main__":  
    port = int(os.environ.get('PORT', 7860))  
    app.run(host='0.0.0.0', debug=False, port=port)
```

Deploy to Hugging Face Spaces:

- **Step 1** — Go to huggingface.co → New Space → Select Docker → Blank template
- **Step 2** — Upload: app.py, Dockerfile, requirements.txt, rdf.pkl, scale1.pkl
- **Step 3** — Upload templates/ folder (home.html, input.html, output.html)
- **Step 4** — Click Commit changes to main — Hugging Face auto-builds and deploys
- **Step 5** — Wait 3–5 minutes for build to complete → App tab shows "Running" The application is now permanently accessible at:

<https://Pradeepkumar143-credit-card-approval.hf.space>

Important Notes:

- **debug=False** is required in app.py to ensure stable operation on cloud platforms.
- **host='0.0.0.0'** is required in app.py to make the app accessible from outside the container.
- **PORT=7860** is the required port for Hugging Face Spaces Docker deployments.
- **rdf.pkl** and **scale1.pkl** must be retrained and saved with the same library versions as specified in requirements.txt to avoid ModuleNotFoundError.

Milestone 7: Conclusion

The **Credit Card Approval Prediction Using IBM Watson Machine Learning** project successfully demonstrates how Machine Learning can automate and improve the credit card approval process in financial institutions. By utilizing historical applicant information and credit history records, the system predicts whether an applicant is likely to be approved or rejected with high accuracy, reducing the need for time-consuming manual verification.

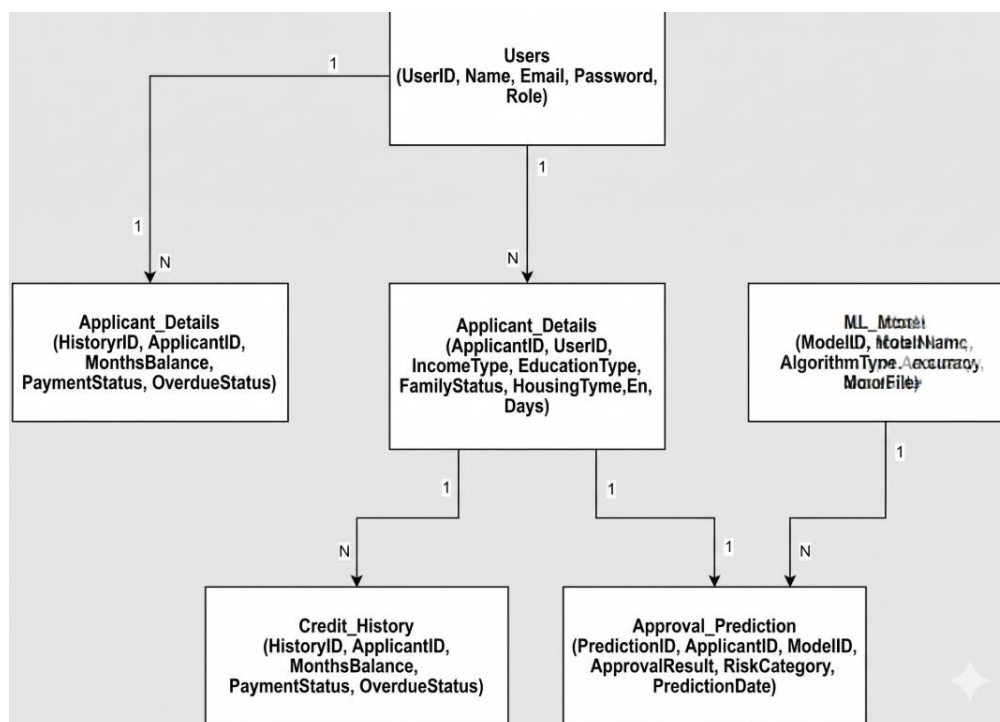
The project begins with comprehensive data preprocessing, including handling missing values, removing duplicate records, feature engineering, and encoding categorical variables to prepare the dataset for model training. Exploratory Data Analysis (EDA) is performed to understand the distribution of applicant attributes and identify meaningful relationships among features that influence credit card approval decisions.

Multiple supervised Machine Learning algorithms, including **Logistic Regression**, **Decision Tree**, and **Random Forest**, are implemented and evaluated using performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix. The best-performing model is selected, saved, and integrated into

a **Flask web application**, enabling users to enter applicant details and receive real-time credit card approval predictions through a simple and user-friendly interface.

The trained model is further deployed using **IBM Watson Machine Learning**, providing a secure, scalable, and cloud-based prediction service. This deployment enables financial institutions to access the prediction model remotely while ensuring faster decision-making and improved operational efficiency. Overall, the proposed system offers an intelligent, reliable, and efficient solution for automating credit card approval decisions. It minimizes manual effort, reduces the possibility of human error, improves prediction accuracy, and supports better risk management in banking and financial organizations. The project also demonstrates the practical application of Machine Learning, Flask, and IBM Cloud technologies in solving real-world financial problems.

Database Design — ER Diagram:



Description:

The Entity Relationship (ER) Diagram represents the database structure of the Credit Card Approval Prediction Using Machine Learning system. It illustrates how applicant information, credit history, machine learning models, and prediction results are interconnected to support the credit card approval process.

The Users entity stores user account information, including UserID, Name, Email, Password, and Role. Each registered user can submit one or more credit card applications, establishing a one-to-many (1:N) relationship between the Users and Applicant_Details entities.

The Applicant_Details entity contains the applicant's demographic and financial information such as ApplicantID, UserID, Income Type, Education Type, Family Status, Housing Type, Employment Days, and other applicant attributes. These details serve as the primary input for the machine learning model to evaluate credit eligibility.

The Credit_History entity maintains the applicant's historical financial records, including HistoryID, ApplicantID, Months Balance, Payment Status, and Overdue Status. Each applicant may have multiple credit history records, creating a one-to-many (1:N) relationship between Applicant_Details and Credit_History. These records help the model understand the applicant's previous financial behavior and repayment performance.

The ML_Model entity stores information about the machine learning algorithms used in the system, including ModelID, Model Name, Algorithm Type, Accuracy, and Model File. This allows multiple trained models to be maintained and compared based on their predictive performance.

The Approval_Prediction entity records the prediction results generated by the selected machine learning model. It contains fields such as PredictionID, ApplicantID, ModelID, Approval Result, Risk Category, and Prediction Date. This entity establishes relationships with both Applicant_Details and ML_Model, ensuring that each prediction is associated with a specific applicant and the machine learning model responsible for generating the result.

Overall, the ER diagram demonstrates a well-structured relational database design that supports applicant data management, credit history tracking, machine learning model integration, and secure storage of prediction results. This architecture ensures efficient data retrieval, maintains referential integrity, and provides scalability for future enhancements to the credit card approval prediction system.